

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
Государственное бюджетное профессиональное
образовательное учреждение города Москвы
«МОСКОВСКИЙ КОЛЛЕДЖ БИЗНЕС-ТЕХНОЛОГИЙ»
(ГБПОУ КБТ)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме: Внедрение системы автоматизации развертывания, масштабирования и управления контейнеризованными приложениями в ЗАО «Эвалар»

СПЕЦИАЛЬНОСТЬ: 09.02.06 «Сетевое и системное администрирование»

ГРУППА: СА41-19

ИСПОЛНИТЕЛЬ:

Чучканов Т. С.

РУКОВОДИТЕЛЬ:

Подвопросом Н. Н.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ, МЕТОДОЛОГИИ И ТЕХНОЛОГИИ.....	4
1.1 Методология DevOps и ее роль в современном мире.....	4
1.2 Контейнеризация и виртуализация. Сферы применения и основные технологии.....	8
1.3 Автоматизация управления конфигурацией. Понятие инфраструктуры как кода.....	12
ГЛАВА 2. МОДЕРНИЗАЦИЯ ИНФРАСТРУКТУРЫ КОМПАНИИ ЗАО «ЭВАЛАР». СОЗДАНИЕ КУЛЬТУРЫ DEVOPS.....	15
2.1 Анализ имеющейся инфраструктуры и подхода к работе. Выделение проблем.....	15
2.2 Поиск способов решения выделенных проблем. Разработка плана модернизации.....	20
2.3 Начало реализации плана по модернизации инфраструктуры.....	28

ВВЕДЕНИЕ

ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ, МЕТОДОЛОГИИ И ТЕХНОЛОГИИ

1.1 Методология DevOps и ее роль в современном мире

Для лучшего понимания дальнейшего материала, представленного в данной выпускной квалификационной работе, для начала раскроем ряд используемых понятий.

DevOps — (от англ. Development и Operations) это набор практик и инструментов, подход к работе, который позволяет компании наладить эффективное взаимодействие сотрудников, занимающихся разработкой ПО, с другими IT-специалистами. Это позволяет производить создание и улучшение продуктов быстрее, чем это происходит, используя более старые и распространенные подходы к разработке программных продуктов.[1][2]

Если рассматривать с точки зрения инструментов, то суть этой методологии заключается в автоматизации и интегрировании рабочих процессов между командами разработки и другими командами IT.

Относительно же взаимодействия между командами, они более не являются «изолированными». То есть члены различных команд отдела IT, а именно, разработки и администрирования, более тесно взаимодействуют друг с другом, как если бы они были членами одной команды. Более того, в некоторых случаях эти команды действительно совмещают в одну команду, где каждый имеет навыки из нескольких дисциплин.

Делается это для исключения проблем, вызываемых разрозненной работой команд, приводящих к замедлению работы в целом, а также для наоборот ускорения, так как процессы работы над разрабатываемыми продуктами и их поставки конечному пользователю являются непрерывными благодаря различным средствам автоматизации процессов.

Подобный подход может включать и взаимодействие с другими командами. Так, например, при тесной интеграции с командой безопасности, получается DevSecOps, где информационная безопасность занимает центральную роль, и все процессы организуются с расчетом на защищенность.

Тем не менее, инструменты не являются основным элементом DevOps. Самая важная основополагающая культуры DevOps — это именно подход к взаимодействию сотрудников. Столь сильная согласованность и слаженность работы людей разных направлений требует мотивации и стараний со всех сторон для достижения максимального эффекта, ведь это требует пересматривания уже устоявшихся привычных вещей.

С ростом популярности и востребованности такой методологии начали появляться и новые специалисты, такие как DevOps-инженеры, занимающиеся налаживанием непрерывности процессов между разработчиками и администраторами.

Так называемый «жизненный цикл DevOps» можно описать как продолжительное циклическое выполнение действий восьми непрерывных этапов*:

- 1) Планирование (Plan);
- 2) Создание/разработка (Code/Create);
- 3) Сборка (Build);
- 4) Тестирование (Test/Validate);
- 5) Выпуск/публикация (Release);
- 6) Развертывание (Deploy);
- 7) Эксплуатация (Operate);
- 8) Мониторинг (Monitor).

Для удобства восприятия данный цикл зачастую изображается в виде всех этапов, расположенных на символе бесконечности (∞). На Рисунке 1 представлен один из примеров подобных схем.

*Это один из множества вариаций описания цикла DevOps. Могут быть незначительные модификации.



Рисунок 1 — Пример схемы, визуализирующей этапы жизненного цикла DevOps, найденной в глобальной сети Интернет.

В ходе выполнения работы был осуществлен анализ отчета «State of DevOps Report»[6] компании Puppet Labs Limited за 2021, публикуемый раз в несколько лет на основе производимых исследований (см. Рисунок 2).

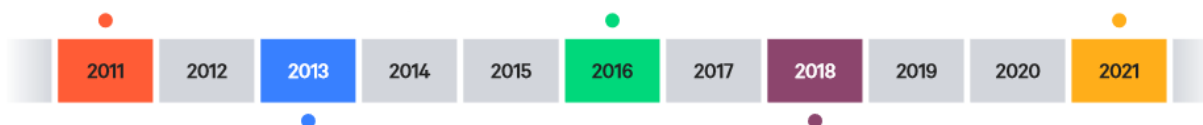


Рисунок 2 — Годы публикации отчетов исследования

Согласно этому отчету, внедрение DevOps лучше всего происходит по модели снизу вверх, начиная с самих команд, и распространяясь на всю организацию. Без поддержки со стороны верхних руководящих лиц компании, компании не получится сильно эволюционировать относительно применяемых подходов к осуществлению рабочей деятельности.

Также компании, внедряющие DevOps, можно условно разделить на три категории по уровню эволюции: низкий, средний, высокий. На разных этапах такой эволюции выделяются схожие факторы, препятствующие дальнейшему процессу модернизации. Среди них можно выделить:

- Неприязнь изменений со стороны коллектива/руководства;
- Старая архитектура;
- Нехватка навыков;

- Ограниченная автоматизация;
- Неясные цели.

Причем, чем выше компания по уровню эволюции принятия DevOps, тем меньше барьеров связаны с культурой внутри компании, и с неясностью целей. Основными барьерами остаются только старая архитектура и нехватка навыков, которые со временем тоже устраняются.

Многие компании останавливаются на среднем уровне внедрения DevOps.

Более подробное рассмотрение данного отчета выходит за рамки этой работы. Поскольку он находится в свободном доступе, ознакомиться с ним может любой желающий в любое время.

1.2 Контейнеризация и виртуализация. Сферы применения и основные технологии

Разберем понятия виртуализации компьютерных систем (виртуальные машины) и контейнеризации, их различия и базовые принципы работы, почему оба подхода к виртуализации популярны. После этого выделим, за счет чего контейнеризация вытеснила виртуальные машины в некоторых сферах, и почему это хорошо для индустрии в целом.

Виртуализация — это процесс более эффективного использования вычислительных ресурсов компьютерной системы за счет разделения этих ресурсов путем применения дополнительных уровней абстракции.

Виртуальные машины — виртуальное представление, или эмуляция, физических компьютерных систем со своими виртуальными компонентами, такими как центральный процессор или оперативная память. Они не имеют возможности напрямую взаимодействовать с физическим компьютером, на котором развернуты. Вместо этого такое происходит при помощи гипервизора.

Гипервизор — технология, которая служит уровнем абстракции между виртуальными машинами (гостями) и нижележащей компьютерной системой (хостом). Гипервизоры выделяют ресурсы компьютерной системы гостевым системам по необходимости.

Существует два вида гипервизоров: первого типа и второго.

Гипервизоры первого типа — легковесные операционные системы, имеющие лишь компоненты, необходимые для обеспечения виртуализации. Они не могут быть использованы как обычные серверные или настольные операционные системы.

Типичные примеры:

- Linux с модулем KVM (например, Proxmox VE, использующий qemu);
- VMWare ESXi;
- Xen;

- Hyper-V.

Гипервизоры второго типа — специальные программы, устанавливаемые поверх существующей операционной системы. Их можно установить на обычные рабочие станции, не влияя на возможность выполнения ежедневных задач.

Типичные примеры:

- VMWare Workstation;
- VirtualBox;
- OpenVZ.

За счет отсутствия лишних компонентов, и некоторых других аспектов гипервизоры первого типа более производительны, и зачастую гостевые системы работают с производительностью очень близко сопоставимой с той, которую можно было бы достичь при использовании системы не как гостевой, а на компьютерной системе непосредственно. Благодаря этому такие гипервизоры используются для развертывания множества виртуализированных серверов. Это позволяет компаниям реализовывать более гибкую и защищенную инфраструктуру, и использовать максимум имеющихся вычислительных ресурсов с минимальным простоем.

Контейнеризация — один из видов легковесной виртуализации на уровне операционной системы или приложения, который работает путем изоляции одного или нескольких процессов. В данном контексте слово «контейнеризация» будет использоваться для описания контейнеризации операционных систем, но не приложений, и относительно операционных систем на базе ядра Linux. Стоит упомянуть, что в других системах имеются схожие технологии. Так, в ОС FreeBSD имеются так называемые «Jails», что в дословном переводе означает «тюрьмы».

Отличие контейнеров от виртуальных машин в том, что они используют одно ядро — операционной системы, на которой запущены. Невозможно запустить контейнер с системой процессорной архитектурой, отличной от

архитектуры системы хоста, на котором разворачивается контейнер. Это же применимо и к самой операционной системе. Это означает, что на компьютере с установленным дистрибутивом Fedora для процессорной архитектуры x86_64 возможно запустить контейнер с Debian или любым другим дистрибутивом для этой архитектуры. Запустить же на такой системе контейнер FreeBSD или для любой другой архитектуры, такой как ARM64 невозможно.

Контейнеры создаются из неизменяемых образов, имеющих файловые системы только для чтения, и содержат только приложение (или несколько), которые планируется разворачивать с их помощью, а также только самые необходимые программы и библиотеки, нужные для работы этого приложения.

Для создания и запуска контейнеров используются низкоуровневые среды выполнения — «рантаймы» (от англ. runtime), используемые более высокоуровневыми средами выполнения, которые добавляют функционал для работы с образами контейнеров, сетевые возможности, и многое другое. Для простого обращения с такими средами выполнения разрабатываются специализированные инструменты, позволяющие разработчикам и системным администраторам работать с контейнерами, не углубляясь в технические сложности.

Существует множество различных сред выполнения контейнеров, рассчитанных на разные сценарии применения, и имеющие разную степень защиты. Одни из самых распространенных низкоуровневых сред выполнения контейнеров — runc и LXC, а среди более высокоуровневых — containerd, CRI-O и LXD. Самыми распространенными высокоуровневыми инструментами для работы с контейнерами являются Docker и Podman.

Тем не менее, вне зависимости от конкретной реализации сред выполнения контейнеров, все они базируются на двух специфических принципах — namespaces и cgroups.

Namespaces (пространства имен) — это абстракции над глобальными ресурсами операционной системы, разделяющие и изолирующие их. Доступ к

ресурсам внутри изолированных пространств имен есть у процессо-участников этих пространств, но не у других процессов. Это позволяет процессам иметь, например, собственные изолированные сетевые интерфейсы, такие как `looback`, либо идентификаторы процессов.

`cgroups` (от англ. `control groups`, контрольные группы) — механизм ядра Linux для иерархической организации процессов и контролируемого, конфигурируемого распределения системных ресурсов в этой иерархии. Контрольные группы образуют древовидную структуру, и каждый процесс в системе одновременно может принадлежать только к одной контрольной группе. Все потоки и дочерние процессы принадлежат той же контрольной группе, что и родительский процесс. При этом, возможно выполнение миграции процесса в другую контрольную группу.

Суммарно, пространства имен — механизм, позволяющий ограничить видимость ресурсов, а контрольные группы ограничивают, какие ресурсы и в каком количестве могут быть использованы.

В современном мире, среди множества других способов применения, контейнеры также используются для: развертывания сред разработки (`Development containers` — контейнеры разработки), временных сред сборки ПО, сред для работы конечных программных продуктов, и так далее. Поскольку контейнеризованный подход к развертыванию программного обеспечения позволяет быстро и удобно разворачивать новые версии ПО, и удалять старые, собирать и распространять образы, компании и отдельные люди все чаще склоняются к использованию контейнеризованного серверного ПО. Где раньше потенциально потребовалось бы каждый раз вручную устанавливать и конфигурировать приложения, теперь можно за несколько секунд получить такой же результат от пары команд по развертыванию контейнера. Также развитие этой технологии открыло и новые возможности, дало толчок к появлению новых технологий, использующих контейнеры как один из главных компонентов.

В некоторых областях виртуальные машины были вытеснены контейнерами. Но полное вытеснение маловероятно, так как эти два подхода к виртуализации нацелены на разные задачи, и скорее дополняют друг друга, чем конкурируют.

1.3 Автоматизация управления конфигурацией. Понятие инфраструктуры как кода

При традиционном подходе, IT-команды полагаются на ручное развертывание инфраструктуры и ручную конфигурацию этой инфраструктуры, требующие совершения большого количества монотонных однообразных действий. Такой подход является неэффективным и легко подверженным ошибкам, resultирующим в увеличенном простое и, соответственно, нарушении работы бизнеса, что в свою очередь зачастую ведет к потерям в денежном эквиваленте.

В современном мире, с каждым годом сложность применяемых в компаниях систем и нагрузка на них постоянно растут, а вместе с ними и количество конфигураций, которые необходимо реализовывать, наряду с увеличением инфраструктуры. Требования к масштабируемости инфраструктуры и эффективности работы с ней также продолжают расти.

Когда администраторам приходится обслуживать большое количество компьютерных систем и сервисов, работающих на них, зачастую превосходящее по размерам само количество обслуживающего персонала, и часто вносить изменения или осуществлять действия по развертыванию новой инфраструктуры, вероятность возникновения ошибок повышается. Эффективность работы же, напротив, уменьшается. Это требует создания новых инновационных подходов к автоматизации.

Постепенно создаются новые методы решения таких проблем. Одними из самых влиятельных являются IaC (Infrastructure as Code, рус. —

инфраструктура как код) и Configuration Management (управление конфигурациями).

IaC — это современный подход к управлению IT-инфраструктурой, при котором инфраструктура описывается при помощи кода.

В отличие от привычного подхода, при котором все изменения и настройки осуществляются администраторами вручную и распределены по разным объектам инфраструктуры, IaC позволяет описывать ее состояние в более унифицированном и контролируемом виде, а затем автоматизированно выполнять развертывание и конфигурацию для достижения этого состояния, используя системы развертывания и системы управления конфигурацией.

Как правило, конфигурационные файлы и файлы с описанием требуемого состояния хранятся в специально отведенных репозиториях, или другими словами хранилищах кода, систем управления версиями, что позволяет иметь единый источник достоверной конфигурации, и отслеживать историю изменений, а также осуществлять восстановление к предыдущим версиям по необходимости.

Благодаря этому, даже самые сложные инфраструктуры могут быть созданы и сконфигурированы быстро и незатруднительно.

В то время, как одни инструменты специализируются на автоматизации развертывания инфраструктуры, другие разработаны для ее конфигурации, но встречаются решения, которые совмещают в себе оба функционала. Так, Terraform и Pulumi — типичные представители инструментов по развертыванию инфраструктуры. Ansible, Puppet, Chef — системы управления конфигурацией.

Использование подобных инструментов направлено на:

- повышение скорости работы обслуживающего персонала при развертывании и настройке компонентов инфраструктуры;
- уменьшение объемов обслуживаемого персонала, требуемого для работы с инфраструктурой.

- снижение вероятности возникновения возможных ошибок, вызванных человеческим фактором;
- обеспечение возможности применения практик разработки ПО к управлению конфигурацией инфраструктуры;
- значительное упрощение развертывания и настройки сложных систем;
- возможность версионирования инфраструктуры и отслеживания изменений путем применения систем управления версиями;
- избежание неоднородности подхода к конфигурации;
- поддержание заданного состояния инфраструктуры;
- повторное использование элементов конфигурации на других компонентах инфраструктуры;
- а также многое другое.

ГЛАВА 2. МОДЕРНИЗАЦИЯ ИНФРАСТРУКТУРЫ КОМПАНИИ ЗАО «ЭВАЛАР». СОЗДАНИЕ КУЛЬТУРЫ DEVOPS

2.1 Анализ имеющейся инфраструктуры и подхода к работе. Выделение проблем

После ознакомления с базовыми терминами и их определениями, а также видами технологий, с которыми предстоит встретиться во время работы с инфраструктурой, можно приступить непосредственно к рассмотрению текущего состояния инфраструктуры компании ЗАО «Эвалар» и подхода членов команд ИТ-отдела к поддержке имеющейся инфраструктуры и развертыванию новой.

В ходе прохождения преддипломной производственной практики был осуществлен сбор информации об имеющейся инфраструктуре и подходах к работе с ней. Получение информации осуществлялось как путем наблюдения за процессом работы некоторых сотрудников, так и при живом опросе некоторых членов команды, в ходе которого была проведена дискуссия со следующими сотрудниками:

- руководителем группы разработки битрикс;
- заместителем директора по IT-инфраструктуре;
- инженером по поддержке и разработке ПО.

После проведения анализа полученной информации, собранной таким образом, можно выделить несколько ключевых аспектов, которые будут описаны далее.

Один из самых главных среди них — процесс внедрения методологии DevOps в работу компании. А именно, каким образом происходит достижение этой цели на пути модернизации подхода к работе компании — за счет использования каких программных средств и изменений в культуре рабочего коллектива.

Поскольку ЗАО «Эвалар» — старая компания, история которой началась в период до двухтысячного года, и имеет большое множество старых проектов, использующих в своей основополагающей части инструменты и решения порой столь же несовременные, подходы к развертыванию и конфигурации инфраструктуры эволюционируют пропорционально медленно. Когда с каждым годом к старой основе добавляются более новые инновационные решения, а конфигурация различных проектов монотонна и в то же время неоднородна, вопросы масштабируемости, надежности и удобства поддержки становятся все более актуальными.

В компании ведется более десятка активных проектов веб-сайтов, нацеленных на предоставление услуг, и каждый из них имеет разный возраст. Разработка одних была начата раньше, а других позже. В отдельных случаях разница может составлять несколько лет. По этой причине технологии и конфигурации, применяемые данными проектами местами сильно разнятся. Решения некоторых проблем, возникавших в ходе разработки и поддержки этих проектов, требовали и внесения изменений в конфигурацию инфраструктуры, на которой они развернуты. Это неизбежно приводит к расхождениям в имеющихся конфигурациях в тех местах, где их не должно быть, а надстройку новых решений произвести труднее.

Все это привело к необходимости кардинального изменения подходов к работе как со стороны разработчиков, так и со стороны администраторов, занимающихся поддержкой инфраструктуры для разрабатываемых проектов.

Члены команд совместно с руководителями провели анализ существующих решений и пришли к выводу, что внедрение методологии DevOps может стать унифицированным подходом к решению описанных выше проблем.

После принятия решения о постепенном переходе к такой модели работы и согласования с вышестоящим руководством, начался процесс модернизации работы компании.

Поскольку для ведения бизнеса важно сохранять стабильность работающих систем, любые вносимые изменения в цикл разработки и конфигурацию инфраструктуры должны затрагивать лишь определенные аспекты работы, и применяться к среде разработки, после подтверждения стабильности в которой, уже применять к производственной среде.

На данный момент в рассматриваемой компании наблюдается низкий уровень внедрения DevOps, обусловленный некоторыми факторами, описанными ранее в данной работе:

- нехваткой навыков (касаемо методологии и определенных видов инструментов);
- старой архитектурой;
- ограниченной автоматизацией.

Инфраструктура компании выполнена в виде ряда виртуальных машин, развернутых на выделенных серверах в виртуальном ЦОД* на базе дата-центра компании РЕКОНН, расположенного на территории Российской Федерации, в городе Москва.

Веб-страница с описанием услуги виртуального ЦОД выглядит следующим образом:

*Центр обработки данных

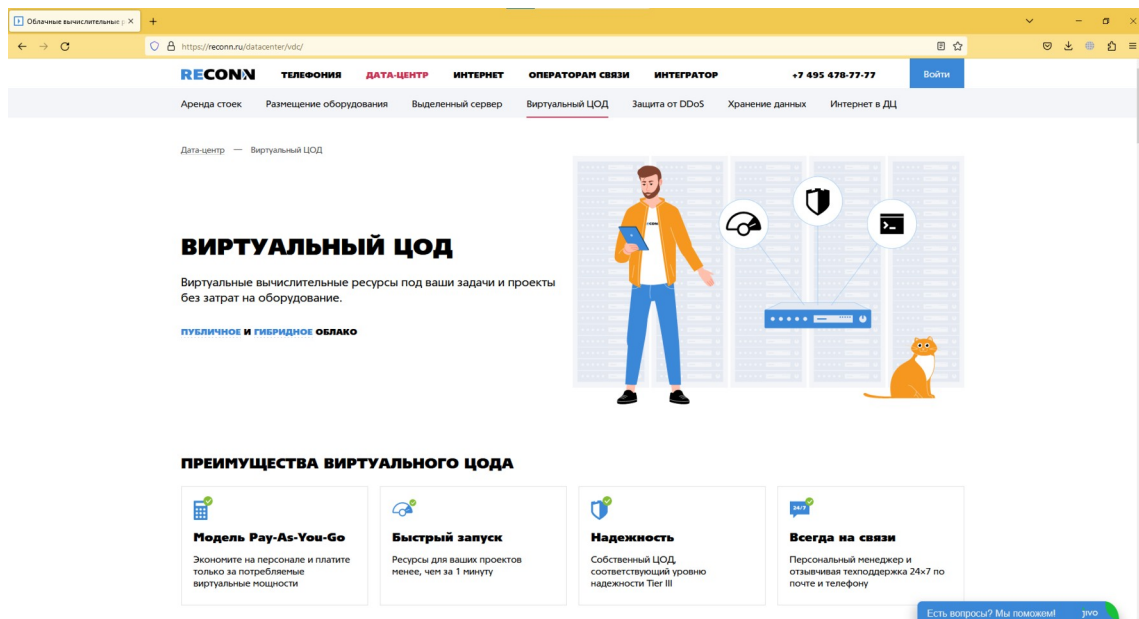


Рисунок 3 — Страница услуги «Виртуальный ЦОД» на сайте РЕКОНН

Виртуальный ЦОД позволяет арендовать необходимое количество вычислительных ресурсов — оперативной памяти, ядер процессора, и так далее. Поскольку при таком подходе администратор компании самостоятельно осуществляет развертывание виртуальных машин с необходимыми ресурсами, это позволяет создать гибкую инфраструктуру по наиболее подходящей к нуждам компании модели.

Как и было сказано ранее, серверы проектов компании выполнены на базе виртуальных машин. Для каждого проекта имеется по две виртуальные машины, одна из которых используется в качестве среды разработки, а другая для производственной среды.

Рабочая нагрузка производственной среды развернута без применения технологий контейнеризации, и автоматизация ее конфигурации минимальна. Среды, используемые разработчиками же, напротив, изолированы в контейнеры. Каждый разработчик имеет доступ к своему контейнеру, и к контейнерам других разработчиков для ситуаций, когда требуется оказать помощь коллегам. Эти среды внутри контейнеров называются стендами разработки. Поскольку могут возникать ситуации, когда отдельный компонент внутри стенда окончательно выходит из строя, либо необходимо развернуть

стенд для нового члена команды, процесс развертывания автоматизирован при помощи CI/CD системы Jenkins. По нажатию кнопки запускается CD «пайплайн», развертывающий и подготавливающий новый стенд разработки. Процесс его развертывания включает в себя такие этапы, как подготовка базы данных веб-сайта при помощи специальных сценариев командной оболочки.

Таким образом, суммируя описанное выше текущее состояние инфраструктуры и методов работы с ней, можно выделить несколько серьезных проблем:

- 1) Ручное и частично автоматизированное развертывание и конфигурация. Виртуальные машины, применяемые для стендов разработки и производственных сред разворачиваются вручную, а их конфигурация осуществляется в полуавтоматическом режиме с применением сценариев командной оболочки. Быстрое создание нескольких виртуальных машин с похожей конфигурацией осуществляется путем развертывания одной виртуальной машины с ее последующим клонированием;
- 2) Низкая портативность. В случае необходимости развертывания идентичной инфраструктуры на базе другого поставщика услуг или собственном сервере, миграция может занять много времени, будет требовать большого количества ручного вмешательства, а также может стать причиной простоя в работе веб-сайтов;
- 3) Излишне повышенная нагрузка обслуживающего персонала. По причине наличия большого количества проектов, обслуживаемых гораздо меньшим количеством администраторов, нагрузка на рабочий коллектив возрастает, что также может привести к неблагоприятным последствиям как для обслуживающего персонала, так и для компании в целом;
- 4) Затрудненность реализации масштабируемости. Такую инфраструктуру невозможно оперативно масштабировать в зависимости от нагрузки и прочих факторов.

5) Отсутствие автоматического поддержания здоровья рабочей нагрузки. При выходе из строя каких-либо сервисов или компонентов, администраторы вынуждены вручную устранять все возникающие типы проблем и повторно приводить сервисы в работоспособное состояние.

2.2 Поиск способов решения выделенных проблем. Разработка плана модернизации

После выделения проблем, которые необходимо решить, можно приступить к созданию плана по модернизации инфраструктуры компании. Необходимо найти подходящие инструменты и методы работы, а также реализовать тестовый вариант модернизированной инфраструктуры для подтверждения предположений, и дальнейшей проработки проекта, чтобы привести его к следующему этапу.

Для начала следует рассмотреть возможные типы инструментов для решения каждой из задач, после чего сопоставить отдельные инструменты и выбрать среди них наиболее подходящие компании по определенным критериям.

Ручное и частично автоматизированное развертывание и конфигурацию инфраструктуры можно заменить автоматизированными системами, такими как Terraform и Pulumi для развертывания, и Ansible, Chef и Puppet для конфигурации.

Низкая портативность, излишняя нагрузка на рабочий персонал, плохая масштабируемость и отсутствие автоматического поддержания состояния здоровья рабочей нагрузки — все эти проблемы могут быть решены применением систем оркестрации контейнеров, такими как Kubernetes, Nomad и Docker Swarm. Этот набор проблем наиболее приоритетен для решения, их необходимо устранить или минимизировать в первую очередь.

Итак, для выбора одной из систем оркестрации контейнеров, составим сравнительную таблицу. Сравнение будет производиться с применением трех

значений веса для определения наличия необходимого функционала: 0 — минимальный или отсутствует, 5 — имеется или достаточно для многих случаев, 10 — имеются широкие возможности. Некоторые ячейки могут содержать вопросительный знак, обозначающий отсутствие достаточного количества информации.

Таблица 1 — Сравнение популярных систем оркестрации контейнеризованных приложений

	Docker Swarm mode	Nomad	Kubernetes
Высокая портативность инфраструктуры с возможностью относительно несложного переноса на другого поставщика или в облако	10	10	10
Открытие дополнительных возможностей автоматизации	5	5	10
Возвращение к предыдущим версиям рабочей нагрузки при возникновении такой необходимости	10	10	10
Широкие возможности по масштабированию рабочей нагрузки и инфраструктуры в целом. В том числе автоматизированно.	5	10	10
Большая устойчивость к проблемам рабочей нагрузки и инфраструктуры за счет самовосстановления	?	10	10
Гибкость выбора отдельных компонентов системы с возможностью их последующей замены	0	0	10
Широкий выбор управляемых облачных решений среди отечественных поставщиков услуг	0	0	10

Продолжение Таблицы 1

Гибкие возможности разграничения доступа по ролям	0	10	10
Открытость системы	10	10	10
Широкая распространенность системы	5	5	10
Исчерпывающая документация	5	10	10
Наличие большого количества обучающих материалов	0	5	10
Zero-downtime разворачивание новых версий и возвращение к предыдущим версиям	10	10	10
Относительная простота архитектуры и скорости проектирования и разворачивания кластеров с нуля	10	10	0

Итого, рассматриваемые системы оркестрации получают следующие баллы:

- 1) Kubernetes — 130;
- 2) Nomad — 105;
- 3) Docker Swarm mode — 70.

Таким образом, наиболее подходящей является система Kubernetes. Поскольку существует множество дистрибутивов этой системы, разрабатываемых разными компаниями и сообществами на базе дистрибутива Kubernetes от Google, называемого «ванильным»^{*} (далее стандартный), далее необходимо выбрать дистрибутив.

^{*}«ванильное» программное обеспечение — не подверженное сторонним модификациям, находящееся в своей изначальной форме. Данный термин широко распространен в компьютерной сфере и пришел из английского языка.

Существует организация Cloud Native Computing Foundation (далее CNCF), одним из множества видов деятельности которой является сертификация дистрибутивов Kubernetes. Для внедрения Kubernetes с целью развертывания критических компонентов инфраструктуры, стоит использовать только сертифицированные дистрибутивы, так как они имеют все необходимые базовые компоненты, которые могут понадобиться для работы, что позволит в том числе менять дистрибутивы при условии отсутствия зависимости от дополнительного функционала конкретного поставщика. Также, такие проекты активно разрабатываются и поддерживаются, что позволит избежать использования проекта, который может перестать существовать через какое-то время после введения в использование.

Существует отдельная страница* сайта CNCF, на которой любой желающий может посмотреть список сертифицированных вариантов Kubernetes. Среди перечисленных решений существуют как облачные управляемые варианты, так и различные установщики для развертывания на собственной инфраструктуре.

Выбор будет осуществлен на основе сопоставления в Таблице 2 среди трех дистрибутивов Kubernetes, предварительно отобранных членами команды. Система оценивания аналогична таковой в Таблице 1.

Таблица 2 — Выбор дистрибутива Kubernetes

	Стандартный Kubernetes от Google	k3s	RKE
Легкость развертывания	0	10	5
Количество сред выполнения контейнеров на выбор	10	5	0
Возможности модификации относительно модульности компонентов	10	0	5

*<https://www.cncf.io/certification/software-conformance/>

Продолжение Таблицы 2

Простота поддержки и администрирования кластера	0	5	5
Качество и доступность документации	10	5	5
Низкая вероятность привязки к поставщику	10	10	10
Популярность согласно субъективным наблюдениям	10	5	0
Низкая или нулевая вероятность отклонения от «ванильного» Kubernetes	10	0	?
Сертифицирован CNCF	10	10	10
Отсутствуют субъективные разработки/предустановки/реализация чего-либо	10	0	0
Низкое потребление ресурсов	0	5	0
Небольшое количество действий/компонентов для минимального рабочего развертывания руками	0	5	5

Итого, рассматриваемые дистрибутивы Kubernetes имеют следующие баллы:

- 1) Стандартный Kubernetes — 80;
- 2) k3s — 60;
- 3) RKE — 45.

Таким образом, наиболее подходящим вариантом является стандартный дистрибутив Kubernetes, разрабатываемый компанией Google.

Далее необходимо выбрать, каким методом будет выполняться развертывание и администрирование кластера, так как для выбранного

дистрибутива на момент создания данной работы существует три инструмента развертывания и администрирования:

- kubeadm;
- kOps
- Kubespray.

Эти инструменты позволяют создавать кластеры, готовые для применения в производственной среде.

Инструмент kOps не поддерживает развертывание кластеров на собственной инфраструктуре и нацелен специфически на использование с поставщиками облачных вычислений, поэтому не подходит.

Kubeadm — это инструмент командной строки, применяемый для установки базовых компонентов кластера на серверы, создания, управления и удаления кластера.

Kubespray — инструмент для работы с кластером, используя Ansible.

Поскольку имеется необходимость перехода на модель IaC, инструмент, который должен применяться для работы с кластером, должен конфигурироваться, используя набор файлов с кодом, описывающим желаемое состояние кластера. Среди двух подходящих инструментов таковым является Kubespray. Дополнительным преимуществом является то, что для использования данного инструмента весь необходимый код клонируется из git репозитория. Это означает, что его можно добавить как подмодуль в другой проект по работе с инфраструктурой, использующий Ansible. Это позволит осуществлять все действия из одного места, а также удобно вести разработку и отслеживать историю изменения всего проекта.

Следующее, что нужно выбрать — инструмент для управления конфигурацией инфраструктуры, который сможет обеспечить применение подхода инфраструктуры как кода. Так как модернизацией инфраструктуры будет заниматься небольшая команда, сильно нагруженная задачами по обеспечению работы сервисов компании, и планируется внедрение сложной

системы автоматизации, развертывания, масштабирования и управления контейнеризованными приложениями, инструмент управления конфигурацией должен быть более легким в изучении, и обеспечивать быстрое начало работы с ним. Поскольку для развертывания и администрирования кластера выбран Kubespray, основанный на базе Ansible, наиболее подходящий вариант — Ansible. Это позволит избежать изучения очередного инструмента, и упростить работу с инфраструктурой, так как она будет выполнена, применяя один инструмент.

Далее потребуется выбор инструмента для автоматизированного развертывания инфраструктуры. Этот инструмент должен обладать исчерпывающей документацией, иметь большое количество поставщиков для развертывания инфраструктуры, а также быть гибким, относительно легким в изучении. Выбор будет осуществлен, используя Таблицу 3. Система оценивания аналогична таковой в Таблице 1.

Таблица 3 — Выбор системы автоматизированного развертывания инфраструктуры

	Terraform	Pulumi
Возможность использование известных администраторам языков программирования	0	10
Наличие большого количества поддерживаемых поставщиков инфраструктуры	10	10
Гибкость описания инфраструктуры	5	10
Возможность осуществлять тестирование конфигурации на ошибки	5	10

Продолжение Таблицы 3

Возможность использования с распространенными редакторами кода и интегрированными средами разработки	5	10
Гибкость описания инфраструктуры, используя специальный язык, возможность реализации сложной логики	5	10

Итого, представленные инструменты получили следующие баллы:

- 1) Pulumi — 60;
- 2) Terraform — 30,

Таким образом, наиболее подходящим инструментом является Pulumi.

Имея список технологий и конкретных реализаций, можно приступить к созданию плана модернизации инфраструктуры компании и создания культуры DevOps среди ее сотрудников.

Для более удобного восприятия план лучше представить в виде пронумерованного списка, так как этапы модернизации должны выполняться последовательно.

- 1) Выбрать проект для тренировки. Перед осуществлением переноса всех проектов стоит выбрать самый простой, состоящий из наименьшего числа компонентов. Это упростит задачу, так как позволит сфокусироваться на углублении понимания принципов работы с Kubernetes вместо траты большого количества времени на попытки запуска отдельных компонентов проекта;
- 2) Спроектировать кластер для окружения разработки, его архитектуру. На этом этапе необходимо понять, из какого количества серверов должен состоять

кластер, какие ресурсы они должен иметь, а также, какие дополнительные компоненты понадобятся для правильной реализации работы проекта, который будет перенесен в первую очередь.;

- 3) Развертывание кластера. После проработки архитектуры, приступить непосредственно к реализации развертывания кластера Kubernetes;
- 4) Осуществить портирование выбранного проекта в Kubernetes. Для этого понадобится выполнить модификации некоторых конфигураций проекта, а также создать новые, необходимые для создания ресурсов API Kubernetes;
- 5) Выполнить перенос стендов разработки. Основываясь на результате портирования проекта, выполнить развертывание стендов разработки;
- 6) Произвести наблюдение за состоянием проекта, периодически собирать информацию от команды разработки для анализа;
- 7) Устранить недочеты, обнаруженные в ходе этапа №6;
- 8) Осуществить этапы №1, №4—6 для оставшихся проектов;
- 9) Внедрить использование инструмента для реализации автоматического развертывания инфраструктуры.
- 10) Продолжительное время, около 3 месяцев осуществлять наблюдение за проектом и устранять возможные недочеты;
- 11) Запросить разрешение у вышестоящего руководства на развертывание производственных сред в кластере Kubernetes;
- 12) Осуществить поэтапно для каждого проекта компании этапы аналогичные этапам №1, №5—7 для развертывания производственных сред.

Итого, имеется план, состоящий из 12 этапов, которые помогут модернизировать инфраструктуру компании ЗАО «Эвалар».

2.3 Начало реализации плана по модернизации инфраструктуры